



react-native从入门到放弃（2）从ios端来看RN运行原理



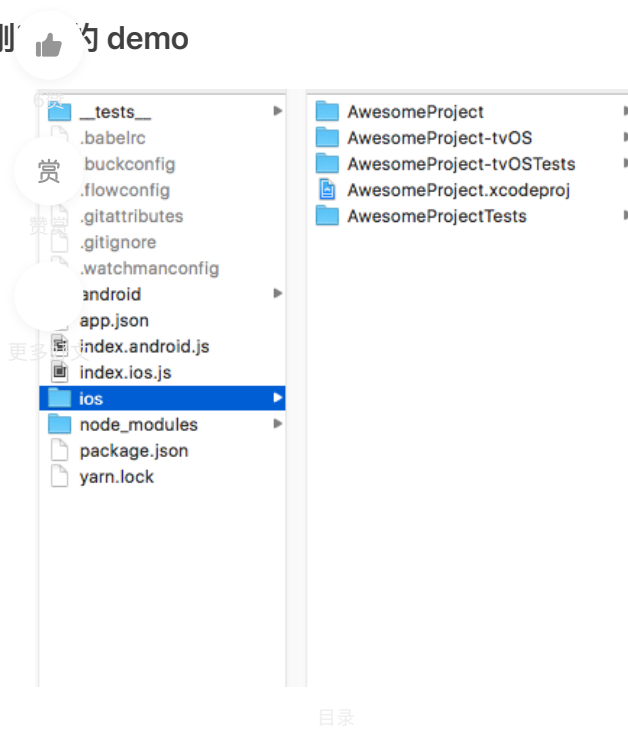
显生宙

关注

0.265

2017.05.09 13:22:35 字数 1,153 阅读 3,162

打开我们刚



分析一下他的实现原理

React Native用iOS自带的JavaScriptCore作为JS的解析引擎，但并没有用到JavaScriptCore提供的一些可以让JS与OC互调的特性，而是自己实现了一套机制，这套机制可以通用于所有JS引擎上，在没有JavaScriptCore的情况下也可以用webview代替，实际上项目里就已经有了用webview作为解析引擎的实现，应该是用于兼容iOS7以下没有JavascriptCore的版本。普通的JS-OC通信实际上很简单，OC向JS传信息有现成的接口，像webview提供的-stringByEvaluatingJavaScriptFromString方法可以直接在当前context上执行一段JS脚本，并且可以获取执行后的返回值，这个返回值就相当于JS向OC传递信息。React Native也是以此为基础，通过各种手段，实现了在OC定义一个模块方法，JS可以直接调用这个模块方法并还可以无缝衔接回调。

RN 与 Objective-C 之间的通信

Objective-C 和 JavaScript 两端都保存了一份配置表，里面标记了所有 Objective-C 暴露给



显生宙

关注

赞赏支持



app程序开发



显生宙

关注

总资产2

外面买的那些外烟都是从哪里来的？是真货吗？

阅读 11

IM及网络相关优秀文献

阅读 458

Mac上安装配置Go语言+Sublime

阅读 751

推荐阅读

JSCore浅析及其在iOS上的使用

阅读 106

iOS与H5交互

阅读 197

Kotlin之flow执行顺序分析（一看就会系列）

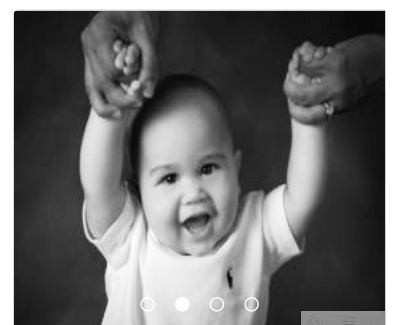
阅读 249

Swift 和OC的block属性不一致问题

阅读 106

大师兄的Python源码学习笔记(三十七)：模块的动态加载机制（四）

阅读 336



儿童弱智的表现

JavaScript 的模块和方法。这样，无论是哪一方调用另一方的方法，实际上传递的数据只有 ModuleId、MethodId 和 Arguments

这三个元素，它们分别表示类、方法和方法参数，当 Objective-C 接收到这三个值后，就可以通过 runtime 唯一确定要调用的是哪个函数，然后调用这个函数。



写下你的评论...

评论0

赞6

...

OC端和JS端分别各有一个bridge，两个bridge都保存了同样一份模块配置表，JS调用OC模块方法时，通过bridge里的配置表把模块方法转为模块ID和方法ID传给OC，OC通过bridge的模块配置表找到对应的方法执行

闭包与回调

既然说到函数互调，那么就不得不提到回调了。对于 Objective-C 来说，执行完 JavaScript 代码再执行 Objective-C 回调毫无难度，难点依然在于 JavaScript 代码调用 Objective-C 之后，如何在 Objective-C 的代码中，回调执行 JavaScript 代码。

目前 React Native 的做法是：在 JavaScript 调用 Objective-C 代码时，注册要回调的 Block，并且把 BlockId

作为参数发送给 Objective-C，Objective-C 收到参数时会创建 Block，调用完 Objective-C 函数后就会执行这个刚刚创建的 Block。

Objective-C 会向 Block 中传入参数和 BlockId

，然后在 Block 内部调用 JavaScript 的方法，随后 JavaScript 查找到当时注册的 Block 并执

看完原理我们来看一下React Native的源码，我的RN Version 是0.44

首先程序入口在AppDelegate中,我去掉了部分源码，我们只看有用的

```
1 | NSURL *jsCodeLocation = [[RCTBundleURLProvider sharedSettings] jsBundleURLForBundleRoot:
2 |
3 | RCTRootView *rootView = [[RCTRootView alloc] initWithBundleURL:jsCodeLocation
4 |                               moduleName:@"AwesomeProject"
5 |                               initialProperties:nil
6 |                               launchOptions:launchOptions];
7 |
```

进入 jsBundleURLForBundleRoot 方法我们来看一下内部的实现

```
1 | - (NSURL *)jsBundleURLForBundleRoot:(NSString *)bundleRoot fallbackResource:(NSString *)
2 | {
3 |     resourceName = resourceName ?: @"main";
4 |     NSString *packagerServerHost = [self packagerServerHost];
5 |     if (!packagerServerHost) {
6 |         return [[NSBundle mainBundle] URLForResource:resourceName withExtension:@"jsbundle"];
7 |     } else {
8 |         NSString *path = [NSString stringWithFormat:@"%@.bundle", bundleRoot];
9 |         // When we support only iOS 8 and above, use queryItems for a better API.
10 |         NSString *query = [NSString stringWithFormat:@"platform=ios&dev=%@&minify=%@",
11 |             [self enableDev] ? @"true" : @"false",
12 |             [self enableMinification] ? @"true" : @"false"];
13 |         return [[self class] resourceURLForResourcePath:path packagerHost:packagerServerHost];
14 |     }
15 | }
16 |
```

这个方法创建了一个本地的js文件访问URL。

继续向下走

```

1  - (instancetype)initWithBundleURL:(NSURL *)bundleURL
2      moduleName:(NSString *)moduleName
3      initialProperties:(NSDictionary *)initialProperties
4      launchOptions:(NSDictionary *)launchOptions
5  {
6      RCTBridge *bridge = [[RCTBridge alloc] initWithBundleURL:bundleURL
7                          moduleProvider:nil
8                          launchOptions:launchOptions];
9
10     return [self initWithBridge:bridge moduleName:moduleName initialProperties:initialPr
11 }
12
13 initWithBundleURL参数分析
14 **bundleURL : 初始化是我们创建的URL
15 moduleProvider : 此块可用于实例化需要额外的模块(这一块我们后期再深入的了解, 先留一个疑问, 暂时不传)
16 launchOptions : 加载的参数**
17

```

注意

此时我们的线程是保持在主线程中执行,在setUp方法中

```

1  void RCTExecuteOnMainQueue(dispatch_block_t block)
2  {
3      if (RCTIsMainQueue()) {
4          block();
5      } else {
6          dispatch_async(dispatch_get_main_queue(), ^{
7              block();
8          });
9      }
10 }
11

```

首先创建了一个RCTBridge对象, 通过文档我们知道, 它是一个异步批量链接JavaScript应用程序通信。

Async batched bridge used to communicate with the JavaScript application.

RCTBridge对象起到了桥接两端的作用,具体实现要在RCTBatchedBridge 类中的 start方法内, start中执行了重要的初始化任务,也是RN的精髓,基于大量的GCD实现了异步初始化组件框架的任务。

```

1  - (void)start
2  {
3      //将要开始加载的通知
4      [[NSNotificationCenter defaultCenter]
5       postNotificationName:RCTJavaScriptWillStartLoadingNotification
6       object:_parentBridge userInfo:@{@"bridge": self}];
7
8      //为事件收集初始事件信息并返回引用ID
9      RCT_PROFILE_BEGIN_EVENT(0, @"-[RCTBatchedBridge setUp]", nil);
10
11     dispatch_queue_t bridgeQueue = dispatch_queue_create("com.facebook.react.RCTBridgeQu
12
13     dispatch_group_t initModulesAndLoadSource = dispatch_group_create();
14
15     // Asynchronously load source code
16     dispatch_group_enter(initModulesAndLoadSource);
17     __weak RCTBatchedBridge *weakSelf = self;
18     __block NSData *sourceCode;
19     // RCTMultipartDataTask 类做网络请求, 基于NSURLSession, 获得javascript data
20     [self loadSource:^(NSError *error, NSData *source, __unused int64_t sourceLength) {
21         if (error) {
22             RCTLogWarn(@"Failed to load source: %@", error);
23             dispatch_async(dispatch_get_main_queue(), ^{
24                 [weakSelf stopLoadingWithError:error];
25             });
26         }
27     }];
28 }
29

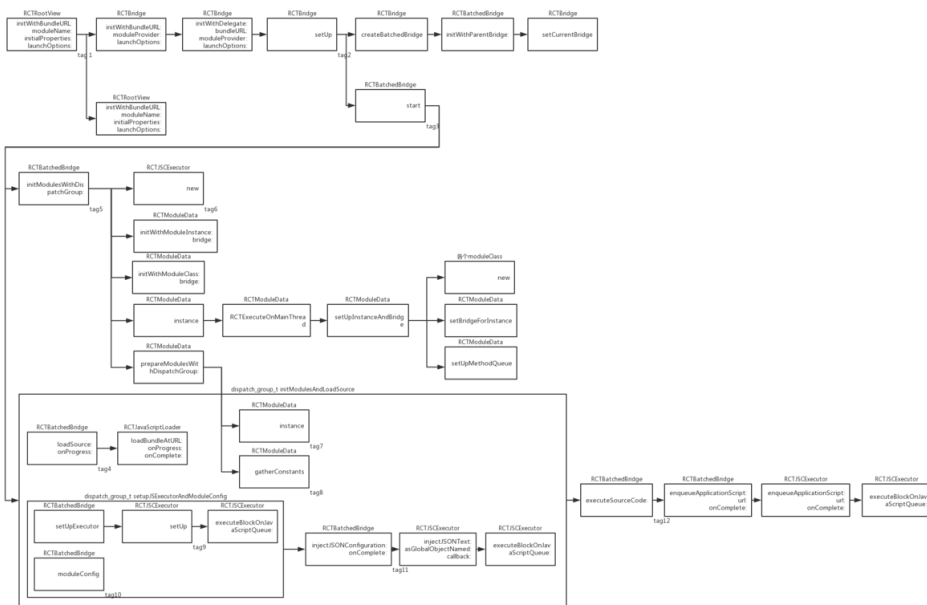
```

```

25     });
26 }
27
28     sourceCode = source;
29     dispatch_group_leave(initModulesAndLoadSource);
30 } onProgress:^(RCTLoadingProgress *progressData) {
31 #if RCT_DEV && __has_include("RCTDevLoadingView.h")
32     RCTDevLoadingView *loadingView = [weakSelf moduleForClass:[RCTDevLoadingView class]
33     [loadingView updateProgress:progressData];
34 #endif
35 }];
36
37 // 同步初始化所有不能被懒加载的本机模块
38 // Synchronously initialize all native modules that cannot be loaded lazily
39 [self initModulesWithDispatchGroup:initModulesAndLoadSource];
40
41 RCTPerformanceLogger *performanceLogger = self->_performanceLogger;
42 __block NSString *config;
43 dispatch_group_enter(initModulesAndLoadSource);
44 dispatch_async(bridgeQueue, ^{
45     dispatch_group_t setupJSExecutorAndModuleConfig = dispatch_group_create();
46
47     // Asynchronously initialize the JS executor
48     dispatch_group_async(setupJSExecutorAndModuleConfig, bridgeQueue, ^{
49         [performanceLogger markStartForTag:RCTPLJSExecutorSetup];
50         [weakSelf setUpExecutor];
51         [performanceLogger markStopForTag:RCTPLJSExecutorSetup];
52     });
53
54     // Asynchronously gather the module config
55     dispatch_group_async(setupJSExecutorAndModuleConfig, bridgeQueue, ^{
56         if (weakSelf.valid) {
57             RCT_PROFILE_BEGIN_EVENT(0, @"-[RCTBatchedBridge moduleConfig]", nil);
58             [performanceLogger markStartForTag:RCTPLNativeModulePrepareConfig];
59             config = [weakSelf moduleConfig];
60             [performanceLogger markStopForTag:RCTPLNativeModulePrepareConfig];
61             RCT_PROFILE_END_EVENT(RCTProfileTagAlways, @"");
62         }
63     });
64
65     dispatch_group_notify(setupJSExecutorAndModuleConfig, bridgeQueue, ^{
66         // We're not waiting for this to complete to leave dispatch group, since
67         // injectJSONConfiguration and executeSourceCode will schedule operations
68         // on the same queue anyway.
69         //利用 javascriptCore 引擎为 native 设置方法和属性
70         [performanceLogger markStartForTag:RCTPLNativeModuleInjectConfig];
71         [weakSelf injectJSONConfiguration:config onComplete:^(NSError *error) {
72             [performanceLogger markStopForTag:RCTPLNativeModuleInjectConfig];
73             if (error) {
74                 RCTLogWarn(@"Failed to inject config: %@", error);
75                 dispatch_async(dispatch_get_main_queue(), ^{
76                     [weakSelf stopLoadingWithError:error];
77                 });
78             }
79         }];
80         dispatch_group_leave(initModulesAndLoadSource);
81     });
82 }];
83
84 //拿到请求后的javascript data 进行解码
85 dispatch_group_notify(initModulesAndLoadSource, bridgeQueue, ^{
86     RCTBatchedBridge *strongSelf = weakSelf;
87     if (sourceCode && strongSelf.loading) {
88         [strongSelf executeSourceCode:sourceCode];
89     }
90 }];
91
92 RCT_PROFILE_END_EVENT(RCTProfileTagAlways, @"");
93 }
94

```

基本流程图



RN启动流程.png

启动流程部分大概就这样，理解了从加载URL/local path 到 通过 jsCore引擎来加载到应用层的基本逻辑，后面继续探索他的原理以及其他的使用技巧，理解原理后才能更好的去使用RN
下面就是将RN应用到自己的项目中开始AwesomeProject 之旅

有问题欢迎评论指正，我会及时进行修改。

参考文章

[React Native 知识库](#)

[React Native通信机制详解](#)

[React Native 从入门到原理](#)

[React Native运行原理解析 android](#)

[React-Native 与 iOS原生项目间通信\(已有项目添加RN\)](#)



react-native从入门到放弃



更多精彩内容，就在简书APP



"小礼物走一走，来简书关注我"

赞赏支持

还没有人赞赏，支持一下



显生宙 倘能生存，我当然仍要学习。
Blog: https://link...
总资产2 共写了2.9W字 获得439个赞 共296个粉丝

关注

程序员学习编程的36个网站，超实用!



写下你的评论...

全部评论 0

只看作者

按时间倒序 按时间正序

被以下专题收入，发现更多相似内容



程序人生



react 从ios到re...

推荐阅读

更多精彩内容 >

React Native

React Native 是最近非常火的一个话题，介绍如何利用 React Native 进行开发的文章和书籍多如...



零度_不结冰 阅读 478 评论 0 赞 2

React Native混合开发(iOS)下的数据交互

导语 React Native是一套由 Facebook 开源的跨平台、动态更新的 Javascript 框架，其...



滴嗒嗒 阅读 7,951 评论 5 赞 36

Hello world!

云服务器是什么？云服务器有什么用？

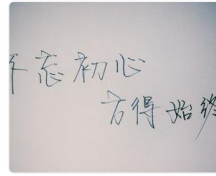


不忘初心，继续写作

初心，百度百科上解释为做某件事的最初的初衷、最初的原因。去年9月份开始加入简书，只停留在看文章的层面，很少动笔。...



四点半的洛杉矶 阅读 196 评论 1 赞 2



扰动

对于一整族群的理想化分析中，会有着无限多的样本，在一个稳定的状态中提炼和简化各式逻辑，用大数定律使得一个确定的世界...



Bolazynes 阅读 226 评论 0 赞 1



培养习惯小窍门

5点早起，晨练，站桩等一系列习惯的养成有两百多天了，经历了冬天的寒冷，夏天的困乏，一直没研究过是这些行为习惯究竟是...



黄洪吉 阅读 78 评论 0 赞 0